Proceedings of the Third International Conference on Electronics and Sustainable Communication Systems (ICESC 2022)
IEEE Xplore Part Number: CFP22V66-ART; ISBN: 978-1-6654-7971-4

Hardware Implementation of Canonical Signed Digit Adder-subtractor Circuit

Satyam Nigam
Electronics and Communication Department
Netaji Subhas University of Technology
Dwarka Sector-3, New Delhi, India
satyamn.sp20@nsut.ac.in

Ayush Mishra
Electronics and Communication Department
Netaji Subhas University of Technology
Dwarka Sector-3, New Delhi, India
ayushm.sp20@nsut.ac.in

2022 3rd International Conference on Electronics and Sustainable Communication Systems (ICESC) | 978-1-6654-7971-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICESC54411.2022.9885336

***Abstract*—Addition and subtraction are two fundamental arithmetic operations performed by modern digital systems such as microprocessors and microcontrollers. A generic hardware design is available for adding two numbers the problem of carry propagation persists. To overcome this issue several attempts were made such as Carry-look ahead adder, Carry save adder, Carry select adder, etc. Previous attempts to optimize the computational complexities are based on a binary number system. This paper prescribes a 4-bit adder circuit for the addition of two numbers in Canonical Signed Digit (CSD) format to optimize the delay and area of adder circuits. The proposed circuit finds its applications in a variety of high-speed ALUs (Arithmetic and logical unit) and digital filters.**

Keywords— Canonical Signed Digit Numbers, Fields Programmable Gate Array, Adder circuits, Carry Save Adder, Carry Select Adder

I. INTRODUCTION

The field of VLSI design always strives for a more optimized hardware implementation of circuits. To help develop these circuits faster, FPGA design flow is often taken into consideration. The FPGA design flow [1] is reliable and fast. In this work an FPGA is used to develop the CSD adder subtractor circuit. This paper taps on the abilities of Canonical signed digit numbers as well as exploits them for performing basic arithmetic operations. The applications of the CSD number system can also be seen in digital FIR filters [2] [3].

Basic adder circuits in digital systems are merely a set of logic gates. These logic gates work on binary and together they are known to be a part of combinational circuit family. First a half adder was developed using one AND gate and one XOR gate. Then by combination of two half adder and an OR gate a full adder circuit was designed. The limitation of such configuration comes into picture when a multibit addition circuit is made by cascading these circuits. These multibit addition circuits are known as Ripple Carry Adders (RCA) [4]. In RC adders, adder of the next stage has to wait for the previous carry in order to calculate the sum. This increases the delay in the calculation of total addition result. The condition gets worse as the size of the inputs increases.

To address this problem several changes were made to optimize the complexity of the adder circuit. One such method was Carry Look-Ahead Adder (CLA) [5]. The CLA adder reduces the propagation delay of the carry by introducing carry generate (G_i) and carry propagate term (P_i). These terms solely depend upon the input bits and by them we can compute carry in advance.

$$\text{Sum} = P_i \oplus C_i$$
$$\text{Carry} = G_i + P_i C_i$$

Above Boolean expressions are the outputs of CLA adder. Where P_i denotes XOR of inputs A_i and B_i and G_i denotes AND operation of A_i and B_i. Though the propagation delay is reduced by CLA adder as the number of bits increases the circuit gets large, asymmetrical and rather complex [6]. Complexity increases the cost of manufacturing as well as area on the chip increases. Carry Save Adders are also a kind of circuits for adding two numbers. This method simply adds n-digit numbers with all the carries generated by adding individual bits of inputs. This saves one carry propagation delay [7]. The carry select adder works on the principle that the carry bit only holds single bit binary value i.e. 0 or 1. Ripple carry adder chain can be replicated for each of these scenarios to save carry propagation. The sum for all possible carry results are precalculated and once the carry value is confirmed correct sum is opted by a multiplexer.

TABLE I. AREA AND DELAY ORDER OF ADDITION STRATEGIES

Addition Strategy	Delay	Area
Ripple Carry Adder	$O(n)$	$O(n)$
Carry Look-Ahead Adder	$O(\log(n))$	$O(n.\log(n))$
Carry Skip Adder	$O(n^{l+2/l+1})$	$O(n)$
Carry Increment Adder	$O(n^{l+2/l+1})$	$O(n)$
Carry Select Adder	$O(n^{l+2/l+1})$	$O(n)$
Carry Save Adder	$O(\log(n))$	$O(n)$

This paper exhibits a new type of adder circuit in which inputs are given in CSD format. The probability of occurrence for a non-zero bit in the canonical SD number system is 0.33 and it is exactly 0.5 for 2'complement numbers. This info helps in understanding the canonical SD number system. Eventually, during addition or subtraction, there will be no carry generation if there are no adjacent non-zero bits. Without using the previous carry we can be able to add two numbers Section II explains this in detail. In Section III Simulink model of CSD adder-subtractor is explained along with the necessary diagrams. The hardware implementation of CSD adder-subtractor circuit is in section IV further discussing the results in section V.

II. ADDITION SUBTRACTION OF TWO CANONICAL SD NUMBERS

A. Canonical Signed Digit Number System

A number involving $\{1,0,\bar{1}\}$ digits are said to be a CSD number. Where $\bar{1}$ represents -1 in decimal format and two digits 1 and 0 represent standard binary. Although several higher radix digits are available [8] we have implemented basic CSD digits for addition and subtraction purpose. To avoid the occurrence of consecutive 1's in a binary number, digit $\{\bar{1}\}$ is used in canonical signed digit number. Binary numbers require to be converted to their equivalent signed digit representation. The recursive algorithm helps in

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:39:46 UTC from IEEE Xplore. Restrictions apply.

retrieving the signed digit representation from a binary number. The principle on which these algorithms is:

If a binary number having l number of 1's and has the value of $2^l - 1$ then same number can be represented by l-1 zeros and only two ones. Further complexity can be reduced by using Higher radix canonical SD numbers can further reduce the complexity of the systems.

TABLE II. 2'S COMPLEMENT AND CSD NUMBER REPRESENTATION

Decimal Value	2's Complement	CSD Equivalent
0	000	000
1	001	001
2	010	010
3	011	$10\bar{1}$
-1	111	$00\bar{1}$
-2	110	$0\bar{1}0$
-3	101	$\bar{1}01$

B. Decimal to CSD conversion

Let us suppose we want to find the CSD equivalent of number $(40731)_{10}$. The binary equivalent of the number would be $(1001111100011011)_2$ and from here we can start converting it to the CSD number by taking the first 2 LSBs and if both are one we can write the number as $100111110001110\bar{1}$ now as we move further examining next two LSBs we can further make alteration such that no non zero digits come consecutive. Finally after several recursions we convert the number to CSD equivalent i.e. $1010000\bar{1}00100\bar{1}0\bar{1}$.

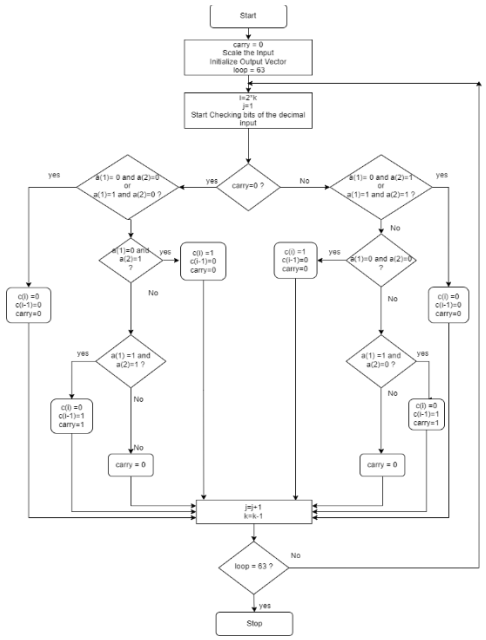


Fig. 1. Conversion algorithm of decimal to encoded CSD number

For a simple canonical SD representation an adder circuit is designed which require one iteration to calculate the final result and saves propagation delay of the carry. Before this part a conversion from decimal to Encoded canonical signed digit format is mandatory. The encoding step is necessary for optimum addition and subtraction operation. One such technique is the negative and positive encoding [9] [10]. This type of encoding technique involves the algorithm which converts 0 to $\langle 0\ 0\rangle$, 1 to $\langle 0\ 1\rangle$ and $\bar{1}$ to $\langle 1\ 0\rangle$ respectively. Rest 1 possibility will be treated as don't care on hardware implementation.

C. CSD Addition Subtraction

Truth table in Table 3 signifies that the canonical signed digit adder consist of combinational hardware logic gates and perform iterations to calculate sum and carry simultaneously. On hardware design a single clock cycle is required to get all these sum and carry values and adder remains ready for next set of inputs for recalculation. Addition module has no dependency over previous carry as Table 3 suggests. Actual novelty resides in the design of canonical SD Adder modules. Decimal to encoded canonical SD number conversion algorithm is applied that checks the stored decimal number in a variable and directly records a encoded CSD code accordingly. Another difference is seen inside the adder block itself as only a set of combinational gates compute the addition and carry simultaneously.

TABLE III. ENCODED CSD NUMBER ADDITION

X_B	X_A	Y_B	Y_A	C_B	C_A	S_B	S_A
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	1	0	0	0	0	0	1
0	1	0	1	0	1	0	0
0	1	1	0	0	0	0	0
1	0	0	0	0	0	1	0
1	0	0	1	0	0	0	0
1	0	1	0	1	0	0	0

The subtraction of two CSD number can be calculated simply as there is no requirement for converting negative number to 2's complement number. In Table 3 $\{X_B, X_A\}$ and $\{Y_B, Y_A\}$ are the encoded form of a single bit inputs. The sum of two bits in the table is calculated in encoded format, $\{S_B, S_A\}$ is the sum of inputs and $\{C_B, C_A\}$ is carry. Hence the last row of Table 3 explains how two negative 1 can be added together to get the result negative 2.

			X_B	X_A
			1	0
			Y_B	Y_A
			1	0
	C_B	C_A	S_B	S_A
	1	0	0	0
Encoded CSD	$\{1$	$0\}$	$\{0$	$0\}$
CSD	$\bar{1}$		0	
Decimal	-2			

Fig. 2. Encoded CSD addition

D. CSD to Decimal conversion

CSD number is converted to decimal using positional notation. In this method, each digit in a CSD number is multiplied to the power of 2. The value of power is decided by the position of digit. Suppose a number is $\bar{1}01$ the LSB i.e. 1 will be multiplied with 2 to the power of zero and further going to other digit, multiplication will be $0x2^1 = 0$ and MSB $\bar{1}$ is known to be as -1 so the multiplication would be $-1*2^2 = -4$. Now adding all the values i.e. -4+0+1= -3. This is the final

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:39:46 UTC from IEEE Xplore. Restrictions apply.

conversion of CSD to binary. Another example is shown below.

$(1100\bar{1}010)_2 = 1x2^7 + 1x2^6 + 0x2^5 + 0x2^4 - 1x2^3 + 0x2^2 + 1x2^1 + 0x2^0 = 128+64+0+0-8+0+2+0 = (186)_{10}$

III. Simulink Model of CSD Adder

A model of CSD adder is prepared to simulate the functionality of the circuit [11] [12]. The model contains combinational logic blocks to add two encoded CSD number. A testbench model for the verification of functionality is used.

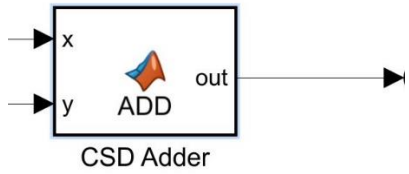


Fig. 3. CSD adder SIMULINK block

In testbench module two 64-bit Decimal to encoded CSD converters are used along with 2 constant decimal input blocks and 1 display block to check the output in decimal. The numbers are given via constant blocks to the decimal to CSD converters from there the converted numbers goes into the CSD Adder block. Further scaling of inputs is used in order to add floating point numbers.

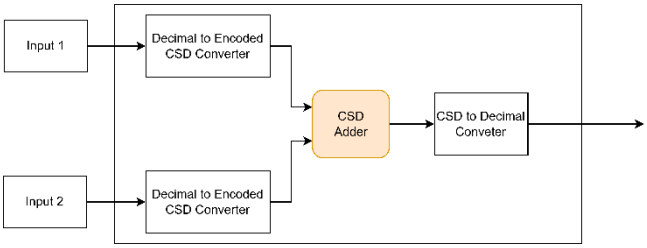


Fig. 4. Testbench module

In Figure 3 the Simulink block shown contains the logical expressions in form MATLAB script in order to mimic the functionality of the adder. Notice that there is no previous carry input in the adder block. This is achieved by taking care carry before adding the numbers. The adder makes a pool vector of both sum and carry simultaneously and add them in a single iteration just as a normal adder would do. Decimal to encoded CSD converter converts the binary number into a encoded CSD number. At the initial stage, role of the converter is crucial in order to maintain the true functionality of the adder-subtractor circuit in the desired number system.

IV. Hardware Implementation of CSD Adder

From Table 3 Boolean expressions for sum (S_A,S_B) and carry (C_A,C_B) outputs. These expressions are implemented using Verilog hardware description language. Here X_A, Y_A, X_B, Y_B are encoded CSD bits.

$$S_A=(\sim X_A)\cdot(\sim Y_B)\cdot(X_A \oplus Y_A)$$
$$S_B=(\sim X_A)\cdot(\sim Y_A)\cdot(X_B \oplus Y_B)$$
$$C_A=(\sim X_B)\cdot X_A\cdot(\sim Y_B)\cdot Y_A$$
$$C_B=(\sim X_A)\cdot X_B\cdot(\sim Y_A)\cdot Y_B$$

TABLE IV. Logic Gate Requirements For Different Adders

S.N.	Type of adder	AND gates	EX-OR gates	OR gates
1	Half-Adder	1	1	-
2	Full Adder	2	2	1
3	8-bit Ripple Carry Adder	16	16	8
4	8-bit Carry look-ahead Adder	16	16	8
5	4-bit Carry save Adder	16	16	8
6	4-bit Carry select Adder	16	16	8
7	4-bit CSD Adder	40	8	-

The FPGA board used to implement the adder circuit is Nexys A7-100T (xc7a100tcsg324-1).

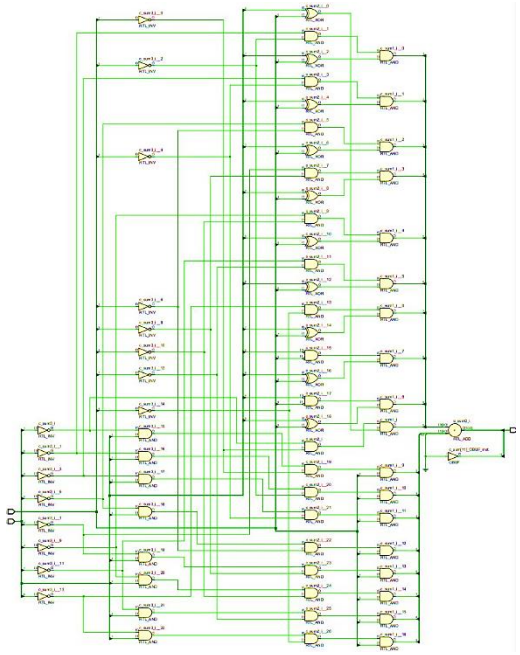


Fig. 5. Schematic of the 4-bit CSD adder

The number of Transistor per bit addition can be calculated for previous as well as prescribed addition strategies. It is evident that total 40 AND gates and 8 EX-OR gates are required to implement a 4-bit CSD adder-subtractor circuit. An EX-OR gate design requires minimum 12 transistors using CMOS design technology. An AND and NOT gates require 6 and 2 transistors respectively for their design [13] [14]. Total 84 Transistor per bit addition is required using Canonical Signed Digit addition.



Fig. 6. Simulation result of 4-bit CSD adder

The post-implementation utilization is based on Xilinx VIVADO EDA tool. Total 18 LUTs and 32 I/O ports are utilized on the Nexys 4 DDR board resulting to total of 0.03% utilization of LUTs and 15.24% utilization of available I/O ports.

TABLE V. FPGA Resource Utilization (Nexys 4 DDR)

	Utilized	% Utilization
LUTs	18	0.03
I/Os	32	15.24

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:39:46 UTC from IEEE Xplore. Restrictions apply.

Delay analysis of three addition strategies is performed.

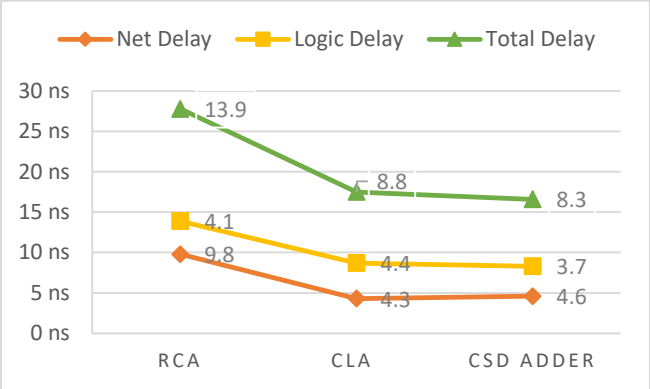


Fig. 7. Delay comparison between different addition stategies

The 4-bit CSD adder has a total delay of 8.3 ns whereas 4-bit ripple carry adder and 4-bit carry look-ahead adder has latency of 13.9 ns and 8.8 ns respectively. The analysis strategies used are mentioned in [15].

V. Conclusion

A unique approach for the addition of two numbers is presented. High speed circuits can utilize the characteristics of canonical signed digit number system. All the results were calculated by Xilinx VIVADO EDA tool. Faster ALUs can be developed by using this work. There is an improvement in data path delay and all timing constraints were met. Carry Look ahead adder has higher path delay as compared to CSD Adder. Signed digit arithmetic have caused improvement in latency whilst maintaining adequate area. Canonical signed digit numbers reduce switching activity and also prevent carry propagation thus overall reducing computational effort of the system. Desirable results were obtained by the design and implementation of Canonical SD Adder-subtractor. The FPGA design flow is used for designing. There is an improvement in area and delay. The carry propagation after addition can be neutralized by the properties of encoded CSD numbers.

References

[1] Chen, Deming, Jason Cong, and Peichen Pan, "FPGA design automation: A survey." Foundations and Trends®," Electronic Design Automation 1, no. 3 (2006): 195-330., (2006).

[2] Hewlitt, Reid M., and E. S. Swartzlantler, "Canonical signed digit representation for FIR digital filters," in *IEEE Workshop on SiGNAL PROCESSING SYSTEMS. SiPS 2000 Design and Implementation* , 2000.

[3] Bria, Oscar N. John G. Proakis and Dimitris G. Manolakis, Digital signal processing. Principles, algorithms, and aplications, Journal of Computer Science and Technology, 1999.

[4] Gurjar, Prashant, Rashmi Solanki, Pooja Kansliwal, and Mahendra Vucha, "VLSI implementation of adders for high speed ALU," in *Annual IEEE India Conference*, 2011.

[5] Pai, Yu-Ting, and Yu-Kumg Chen. , "The fastest carry lookahead adder," in *Second IEEE International Workshop on Electronic Design, Test and Applications*, 2004.

[6] Miao, Jia, and Shuguo Li., "A novel implementation of 4-bit carry look-ahead adder," in *International Conference on Electron Devices and Solid-State Circuits (EDSSC)*, 2017.

[7] R. Hashemian, "A fast carry propagation technique for parallel adders," in *the 33rd Midwest Symposium on Circuits and Systems, pp. 456-459. IEEE.*, 1990.

[8] J. O. Coleman, "Express coefficients in 13-ary, radix-4 CSD to create computationally efficient multiplierless FIR filters," in *Proc. European Conf. on Circuit Theory and Design*, 2001.

[9] B. Parhami, " Carry-free addition of recoded binary signed-digit numbers," in *IEEE Transactions on computers 37, no. 11: 1470-1476*, 1988.

[10] B. Parhami, Computer arithmetic, New York, NY: Oxford university press, 2010.

[11] Karris, Steven T. , Digital circuit analysis and design with Simulink modeling and introduction to CPLDs and FPGAs., Orchard Publications, 2007.

[12] Grout, I., J. Ryan, and T. O'Shea., "Configuration and debug of field programmable gate arrays using MATLAB®/SIMULINK®," *Journal of Physics: Conference Series,. IOP Publishing,* vol. 15, no. 1, p. 041, 2005.

[13] Leblebici, Yusuf, and Sung-Mo Kang. , CMOS digital integrated circuits: analysis and design., New York: McGraw-Hill, 1996.

[14] Aguirre-Hernandez, Mariano, and Monico Linares-Aranda, "CMOS full-adders for energy-efficient arithmetic applications," *IEEE transactions on very large scale integration (VLSI) systems,* vol. 19, no. 4, pp. 718-721, 2010.

[15] P. Karuppusamy, "Design and analysis of low-power. high-speed baugh wooley multiplier," *Journal of Electronics,* vol. 2, pp. 60-70, 2019.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:39:46 UTC from IEEE Xplore. Restrictions apply.